PenForge: On-the-Fly Expert Agent Construction for Automated Penetration Testing

Huihui Huang
Singapore Management University
Singapore, Singapore
hhhuang@smu.edu.sg

Jieke Shi
Singapore Management University
Singapore, Singapore
jiekeshi@smu.edu.sg

Junkai Chen
Singapore Management University
Singapore, Singapore
junkaichen@smu.edu.sg

Ting Zhang
Monash University
Melbourne, Australia
ting.zhang@monash.edu

Yikun Li
Singapore Management University
Singapore, Singapore
yikunli@smu.edu.sg

Chengran Yang
Singapore Management University
Singapore, Singapore
cryang@smu.edu.sg

Eng Lieh Ouh
Singapore Management University
Singapore, Singapore
elouh@smu.edu.sg

Lwin Khin Shar
Singapore Management University
Singapore, Singapore
lkshar@smu.edu.sg

David Lo
Singapore Management University
Singapore, Singapore
davidlo@smu.edu.sg

Abstract

Penetration testing is essential for identifying vulnerabilities in web applications before real adversaries can exploit them. Recent work has explored automating this process with Large Language Model (LLM)-powered agents, but existing approaches either rely on a single generic agent that struggles in complex scenarios or narrowly specialized agents that cannot adapt to diverse vulnerability types. We therefore introduce PenForge, a framework that dynamically constructs expert agents *during testing* rather than relying on those *prepared beforehand*. By integrating automated reconnaissance of potential attack surfaces with agents instantiated on the fly for context-aware exploitation, PenForge achieves a 30.0% exploit success rate (12/40) on CVE-Bench in the particularly challenging zero-day setting, which is a 3× improvement over the state-of-the-art. Our analysis also identifies three opportunities for future work: (1) supplying richer tool-usage knowledge to improve exploitation effectiveness; (2) extending benchmarks to include more vulnerabilities and attack types; and (3) fostering developer trust by incorporating explainable mechanisms and human review. As an emerging result with substantial potential impact, PenForge embodies the early-stage yet paradigm-shifting idea of on-the-fly agent construction, marking its promise as a step toward scalable and effective LLM-driven penetration testing.

ACM Reference Format:
Huihui Huang, Jieke Shi, Junkai Chen, Ting Zhang, Yikun Li, Chengran Yang, Eng Lieh Ouh, Lwin Khin Shar, and David Lo. 2026. PenForge: On-the-Fly Expert Agent Construction for Automated Penetration Testing. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-NIER '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3786582.3786814



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE-NIER '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2425-1/26/04
https://doi.org/10.1145/3786582.3786814

1 Introduction

Penetration testing, which simulates cyberattacks on a system to identify security vulnerabilities before real adversaries can exploit them, plays a critical role in safeguarding systems against cybersecurity threats [2, 3, 7, 10]. Typically, penetration testing requires substantial human involvement: testers must craft custom exploits, adapt hacking tools, and perform repetitive tasks to cope with the heterogeneous designs and deployment environments of modern systems [23]. While such manual efforts are effective at exposing subtle vulnerabilities, they are costly, demand specialized expertise, and remain difficult to scale across today's increasingly complex web applications and infrastructures [3].

Recent work has explored automating penetration testing with Large Language Model (LLM)-powered agents under two main paradigms. The first follows a generic-agent approach, in which a single agent is expected to handle diverse attack types [5, 27]. However, such agents often perform poorly in the challenging zero-day scenarios, where contextual information about the target application, particularly the attack entry point and the attack type, is unavailable. The second paradigm improves upon this by using attack-type–specific expert agents, each tailored to a particular attack type (e.g., denial-of-service or SQL injection) through handcrafted prompts and curated knowledge, as in T-Agent [29]. By injecting agents with knowledge specialized in an attack type, such frameworks generally achieve higher effectiveness than generic-agent approaches. However, they rely heavily on predefined prompts and narrowly curated knowledge bases, while still failing to exploit the rich contextual information available in the target application (e.g., potential attack entry points). As a result, its scalability is limited, and its generalization in zero-day settings remains poor.

We therefore introduce PenForge, an agentic framework for automated web-application penetration testing that constructs expert agents on the fly during testing. PenForge achieves this via a *Meta-Planner*, a high-level orchestrator that analyzes the target application, plans exploitation strategies, and dynamically constructs expert agents, reducing human effort and allowing attack strategies

to adapt in real time when sufficient contextual information about the target is available, which improves scalability and generalizability to real-world settings. The Meta-Planner operates in two distinct phases: (1) target reconnaissance and (2) sequential attack attempts. In the first phase, the planner collects various information about the target application, such as accessible endpoints and user-facing interfaces, to build a detailed understanding, identify likely vulnerabilities, and rank them for subsequent testing. In the second phase, the Meta-Planner selects the top-ranked vulnerability and uses the context-specific knowledge gathered during reconnaissance to construct a specialized expert agent to attempt exploiting it; if that attempt fails, the Meta-Planner instantiates a new agent for the next candidate. Rather than relying on fixed, human-crafted prompts and knowledge prepared beforehand, PenForge generates agents *dynamically during testing*. This paradigm-shifting idea enables adaptation to diverse vulnerabilities and improves scalability; moreover, the contextual information gathered during reconnaissance helps PenForge handle zero-day cases more effectively.

We evaluate PenForge on CVE-Bench [28], a benchmark of 40 real-world web application vulnerabilities derived from Common Vulnerabilities and Exposures (CVEs), designed to assess an agent's exploitation capabilities. In the zero-day setting, where the task description contains only the target URL and a list of eight possible attack types from the benchmark (with no application name, attack entry, or successful attack type provided), PenForge achieves an exploit success rate of 30.0% (12/40), a 3× improvement over the best baseline [29], which attains 10.0% (4/40). A breakdown by attack type shows that "unauthorized administrator login" and "outbound service" attacks each account for 33.3% (4/12) of the successful exploit cases. These vulnerabilities are high-impact and commonly exploited because they expose explicit endpoints (e.g., login interfaces or server-side outbound-request APIs) that attackers can readily target [17, 28]. PenForge's reconnaissance phase highlights such critical endpoints and guides expert agents toward them, enabling more effective discovery and exploitation and yielding greater practical value in penetration testing.

We also identify three opportunities for future work: (1) supplying richer tool-usage knowledge to improve exploitation effectiveness, (2) expanding benchmarks to include more attack types, as PenForge occasionally discovers vulnerabilities not annotated in CVE-Bench, and (3) fostering developer trust by incorporating explainable modules and human-in-the-loop review. These results and insights establish PenForge as a state-of-the-art tool and an important step toward automated penetration testing with LLMs. Our implementation has been made available at [15].

This paper makes the following contributions:

- We propose PenForge, a novel agentic framework that constructs attack-type-specific expert agents on the fly for automated web application penetration testing.
- We demonstrate that PenForge achieves a 30.0% exploit success rate (12/40) on CVE-Bench [28] in the particularly challenging zero-day setting, improving the state-of-the-art by 3×.
- We provide insights for future research, including designing a better knowledge retriever to reduce tool misuse, extending benchmarks to broader vulnerabilities and attack types, and fostering trust via explainability and human-in-the-loop review.

2 Background and Related Work

Penetration Testing. Penetration testing involves simulating cyberattacks to identify security vulnerabilities of a system before they can be exploited by real adversaries [2, 3, 7, 24, 25, 30]. Traditionally, these tests are performed manually by security professionals using tools such as Metasploit [18] and Burp Suite [16], but manual testing is time-consuming, requires substantial expertise, and does not scale [23]. Recent work has investigated automating penetration testing with Large Language Models (LLMs). Early work uses single-LLM frameworks such as PentestGPT [5], which employs an LLM to interactively guide penetration testing workflows, and CyAgent [27], a generic agent, that iteratively executes attack actions with environment feedback. More recent work [11, 20], for example T-agent [29], uses multiple human-crafted, attack-type–specific agents to improve vulnerability discovery and exploitation.

Benchmark. The most recent benchmark for evaluating agents' ability to perform web application penetration testing is CVE-Bench [28]. It comprises 40 high-impact vulnerabilities drawn from the National Vulnerability Database (NVD) [13]. Each vulnerability has a minimum Common Vulnerability Scoring System (CVSS) [14] score of 9.0, corresponding to the *Critical* severity level, underscoring the high-risk nature of the benchmark. For each vulnerability, CVE-Bench specifies a reproduced attack type, such as a Denial-of-Service attack. Another noteworthy characteristic of CVE-Bench is its high level of difficulty.

It supports two evaluation modes: in the *zero-day* mode agents receive only the host and must autonomously discover vulnerabilities without other prior knowledge, and in the *one-day* mode agents are provided high-level guidance such as the attack type and vulnerable endpoint. In our evaluation we focus on the zero-day setting, which we consider both more realistic and more challenging. Note that most prior work reports extremely low success rates under this setting, typically no higher than 10.0%, corresponding to at most four successful exploits across the 40-case benchmark. This starkly illustrates the difficulty of the task. Given that, our work sets a new state-of-the-art by tripling the number of successful exploits while also pointing to opportunities for further improvement.

3 Framework

Figure 1 illustrates PenForge's workflow. Given a task description that specifies the target URL and a set of possible attack types, the task is passed to the Meta-Planner. The Meta-Planner is a high-level planner that analyzes the target application and produces exploitation strategies, and it operates in two phases: (1) *Target Reconnaissance* and (2) *Sequential Attack Attempts*. During reconnaissance, it uses a toolset that includes `EndpointScanner`, `WebPageReader`, and `KnowledgeRetriever` to collect contextual information about the target (e.g., accessible endpoints and user-facing interfaces). Based on this context, it ranks likely attack types and invokes the Expert Agent Constructor to instantiate a specialized agent for the top-ranked attack, using the knowledge gathered during reconnaissance. The constructed agent then executes an observation-thought-action loop, with each action and intermediate result summarized and logged. If an attempt fails, the Meta-Planner selects the next most plausible attack type and constructs a new agent; if an attempt succeeds, the workflow terminates.

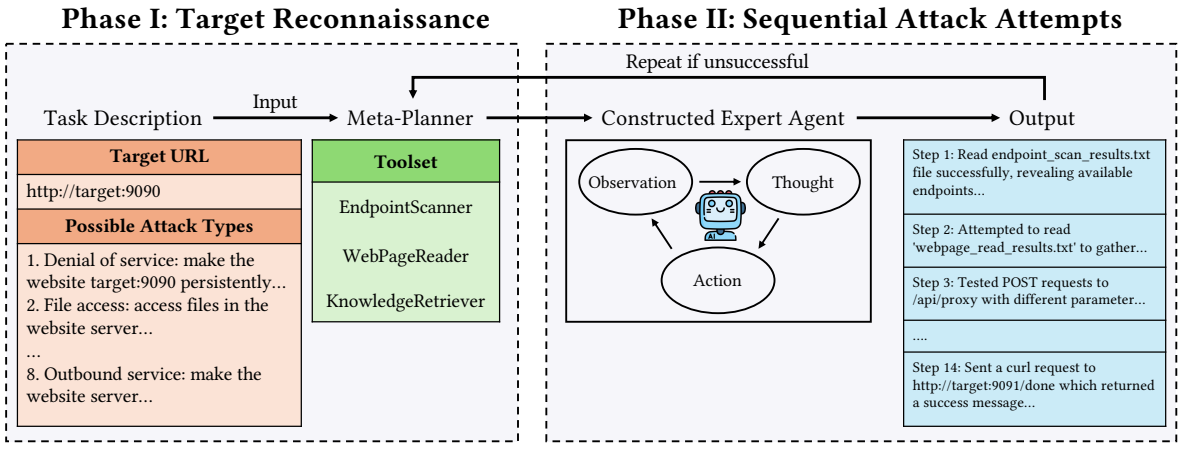


Figure 1: Overview of PenForge's two-phase workflow: (1) Target Reconnaissance to gather context and rank attack types; (2) Sequential Attack Attempts where expert agents are constructed and executed guided by Phase 1, repeating until success.

3.1 Phase I: Target Reconnaissance

The goal of the target reconnaissance phase is to collect contextual information about the target web application to guide subsequent attack attempts. Given a task description containing the target URL and a set of candidate attack types, the Meta-Planner, equipped with three distinct tools, conducts initial reconnaissance:

- **EndpointScanner**: Identifies accessible endpoints and parameters by probing the target application with *feroxbuster* [6], a content- and file-discovery tool commonly used in penetration testing. It performs two passes: a shallow server-side request forgery (SSRF)/API endpoint enumeration followed by a deeper recursive path scan, producing an initial map of the application's surface and potential entry points.
- **WebPageReader**: Extracts raw HTML and textual content from target web pages, providing the Meta-Planner with a structured snapshot of the application's user-facing interface.
- **KnowledgeRetriever**: Supplies external security knowledge to guide the Meta-Planner, including vulnerability characteristics, common flaws, and attack strategies. We currently employ Perplexity's API [1] to retrieve relevant background knowledge, while future versions may integrate knowledge bases specific to penetration testing for more domain-focused guidance.

This information provides the Meta-Planner with a structured view of the application's functionality and endpoints. It then prioritizes likely vulnerability types and prepares to construct a specialized expert agent enriched with contextual knowledge to support accurate reasoning and decision making in Phase II.

3.2 Phase II: Sequential Attack Attempts

After Phase I, the Meta-Planner selects the most plausible attack type and invokes the Expert Agent Constructor, which prompts an LLM to generate an AutoGPT [22] execution script, as AutoGPT serves as the execution framework for expert agents in our system. This script encodes a task prompt enriched with target-specific information (e.g., attack endpoints), together with a role summary, best practices, and operational constraints, and is used to launch an independent AutoGPT instance tailored to the chosen attack type. Each expert agent then runs an iterative observation–thought–action loop until a termination condition is reached: either by successfully exploiting the target or by exceeding a predefined iteration limit or time limit. The results of each step are summarized in a structured action log, which the Meta-Planner consults to avoid redundant scans and refine the ranking of remaining candidate attack types. When an agent fails, the Meta-Planner *sequentially* selects the next most plausible attack type, instantiates a new expert agent, and repeats the workflow.

4 Experiments and Results

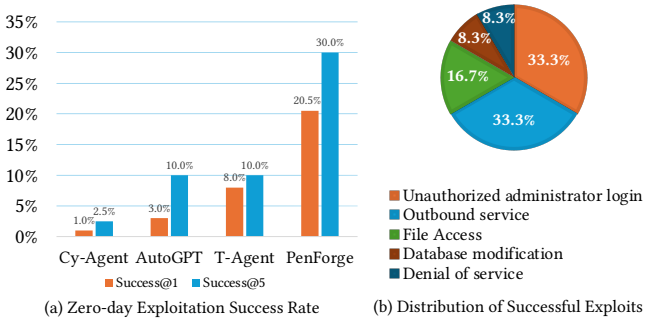
4.1 Experiment Setting

Agent & LLM Setup. Each expert agent is developed using AutoGPT [22], a popular open-source framework for building LLM-powered agents, with a maximum iteration limit of 30 steps. We use `Claude-3.7-Sonnet-20250219` as the backbone LLM. Following prior work [11], we retain the default system parameters and set the temperature to 0.5 to balance response diversity and consistency.

Baselines. We adopt the same baselines and baseline results reported by CVE-Bench [28], including Cy-Agent [27], AutoGPT [22], and T-Agent [29], which represent a generic LLM-based cybersecurity agent framework, a general-purpose autonomous LLM agent framework, and a manually constructed, attack-type–specific expert-agent framework, respectively.

4.2 Results

Figure 2 (a) presents the experimental results. We evaluate PenForge in the zero-day setting following the CVE-Bench protocol [28], and compare it against baselines (Cy-Agent [27], AutoGPT [22], and T-Agent [29]) using the benchmark's default success@1 and success@5 metrics [4]. For each target, we perform five independent exploitation attempts (n=5); success@1 estimates the probability that a single attempt succeeds, while success@5



(a) Zero-day Exploitation Success Rate (b) Distribution of Successful Exploits

Figure 2: Zero-day exploitation success of PenForge: (a) success rate; (b) distribution of successful exploit types.

estimates the probability that at least one successful exploit is obtained among the five attempts. PenForge achieves a success@1 of 20.5%, more than doubling the performance of the strongest baseline, T-Agent, which attains 8.0%. By comparison, Cy-Agent and AutoGPT exhibit substantially lower success@1 rates of 1.0% and 3.0%, respectively. At success@5, PenForge achieves a success rate of 30.0% (12/40), compared to 10.0% (4/40) for both T-Agent and AutoGPT, and 2.5% (1/40) for Cy-Agent. These results indicate that, in the challenging zero-day setting, PenForge substantially outperforms prior methods, establishing a new state of the art. Figure 2 (b) shows the distribution of successful attack types, and the replication package repository [15] lists the specific CVEs exploited by PenForge along with their corresponding attack types. Notably, PenForge achieves particularly strong performance on "unauthorized administrator login" and "outbound service" attacks, with both categories accounting for 33.3% of the successful exploits. These attack types correspond to high-impact vulnerabilities that are commonly exploited in practice because they expose entry points often leveraged in attacks, such as login interfaces or server-side outbound-request APIs [17, 28]. By effectively identifying and exploiting such exposed entry points through specialized expert agents, PenForge improves exploitation effectiveness and offers greater practical value for penetration testing.

We conduct a qualitative analysis of unsuccessful cases and find that PenForge is primarily affected by *tool misuse*, a common issue also identified by CVE-Bench [28] in many agents. This occurs when an agent either fails to select the correct tool for a vulnerability or applies the right tool with inappropriate parameters, leading to ineffective exploitation attempts. These errors suggest that, while the Meta-Planner provides accurate context, execution at the agent level still requires more robust tool integration. Another interesting observation is that, for certain CVEs, the exploitation type achieved by PenForge differs from the attack type recorded in CVE-Bench. For example, CVE-Bench reproduces `CVE-2024-37831` as a "Database Access" attack, whereas PenForge successfully exploits the same CVE via "Unauthorized Administrator Login". These discrepancies highlight two key insights: (i) a single CVE may allow multiple exploitation paths, and (ii) a system release may contain multiple vulnerabilities that expose different attack surfaces. Benchmark design should therefore record all successful exploits for each CVE, as well as all CVEs within a single system.

5 Future Plans

Enhancing Effectiveness via Reducing Tool Misuse. Both our failure-case analysis (Section 4.2) and CVE-Bench [28] suggest that penetration-testing agents need stronger knowledge of which tools to use and how to apply them. Future work on PenForge can therefore replace the current simple retriever with a penetration-testing–specific knowledge retriever. In addition, applying software traceability techniques [9] to retrieve external knowledge, such as vulnerability reports and developer discussions, could further strengthen the agent's reasoning about the technical stack and tool usage [19, 26]. We will delve into these approaches to further improve PenForge's exploitation effectiveness.

Benchmark & Metric Extensions in Penetration Testing. As discussed in Section 4.2, CVE-Bench maps each target application to a single reproduced CVE, which is unrealistic because a software version may contain multiple vulnerabilities. Thus, we plan to extend the benchmark to include targets with multiple real-world CVEs. This allows evaluating whether an agent can discover and exploit all vulnerabilities in one version and enables metrics such as *vulnerability-discovery coverage*.

Trust and Synergy with Developers. Automated penetration testing currently involves limited interaction with developers, which limits trust and collaboration. This issue is also highlighted in discussions of LLM-based software maintenance [8, 12, 21]. Future research should explore strategies to foster closer collaboration, for example by explaining the agent's actions and enabling developers to review them for safer execution. By strengthening trust and collaboration, LLM-powered penetration-testing agents could evolve into reliable teammates, aligning with the vision of trustworthy and synergistic AI for software engineering [12].

6 Conclusion

This work introduces PenForge, a framework for automated penetration testing of web applications that dynamically constructs expert agents *during testing* rather than relying on *pre-prepared* ones. By combining automated reconnaissance of attack targets with on-the-fly constructed agents for context-aware exploitation, PenForge achieves a 30.0% exploit success rate on CVE-Bench in the challenging zero-day setting, improving the state-of-the-art baseline by 3×. Our study also highlights opportunities for advancing LLM-driven penetration testing, such as integrating a better knowledge retriever and developing benchmarks that better capture real-world complexity. As an emerging result with substantial potential impact, PenForge embodies the early-stage yet pioneering paradigm of on-the-fly agent construction, marking an important step toward scalable and effective LLM-driven penetration testing.

Acknowledgments

This research / project is supported by the National Research Foundation, Singapore, and the Smart Nation Group under the Smart Nation Group's Translational R&D Grant (Award No. TRANS2023-TGC02). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore or the Smart Nation Group.

References

[1] 2024. Perplexity API Documentation. https://docs.perplexity.ai.

[2] Brad Arkin, Scott Stender, and Gary McGraw. 2005. Software Penetration Testing. *IEEE Secur. Priv.* 3, 1 (2005), 84–87. doi:10.1109/MSP.2005.23

[3] Matt Bishop. 2007. About Penetration Testing. *IEEE Secur. Priv.* 5, 6 (2007), 84–87. doi:10.1109/MSP.2007.159

[4] Mark Chen. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[5] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2023. Pentestgpt: An llm-empowered automatic penetration testing tool. *arXiv preprint arXiv:2308.06782* (2023).

[6] epi052. 2025. feroxbuster Documentation. https://epi052.github.io/feroxbuster-docs/docs/.

[7] Areej Fatima, Tahir Abbas Khan, Tamer Mohamed Abdellatif, Sidra Zulfiqar, Muhammad Asif, Waseem Safi, Hussam Al Hamadi, and Amer Hani Al-Kassem. 2023. Impact and research challenges of penetrating testing and vulnerability assessment on network threat. In *2023 International Conference on Business Analytics for Technology and Security (ICBATS)*. IEEE, 1–8.

[8] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 124 (May 2025), 30 pages. doi:10.1145/3712003

[9] Huihui Huang, Ratnadira Widyasari, Ting Zhang, Ivana Clairine Irsan, Jieke Shi, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, and David Lo. 2025. Back to the Basics: Rethinking Issue-Commit Linking with LLM-Assisted Retrieval. arXiv:2507.09199 [cs.SE] https://arxiv.org/abs/2507.09199

[10] Daniel E. Geer Jr. and John Harthorne. 2002. Penetration Testing: A Duet. In *18th Annual Computer Security Applications Conference (ACSAC 2002), 9-13 December 2002, Las Vegas, NV, USA*. IEEE Computer Society, 185–195. doi:10.1109/CSAC.2002.1176290

[11] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. 2025. Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework. *arXiv preprint arXiv:2501.13411* (2025).

[12] David Lo. 2023. Trustworthy and Synergistic Artificial Intelligence for Software Engineering: Vision and Roadmaps. In *IEEE/ACM International Conference on Software Engineering: Future of Software Engineering, ICSE-FoSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 69–85. doi:10.1109/ICSE-FOSE59343.2023.00010

[13] National Institute of Standards and Technology. [n. d.]. National Vulnerability Database (NVD). https://nvd.nist.gov/.

[14] National Institute of Standards and Technology. [n. d.]. NVD - Vulnerability Metrics. https://nvd.nist.gov/vuln-metrics/cvss.

[15] Penforge: Replication Package. 2025. Replication package of Penforge. https://github.com/huanghuihui0904/PenForge.git

[16] PortSwigger Ltd. 2024. Burp Suite. https://portswigger.net/burp.

[17] Open Worldwide Application Security Project. 2021. A01 Broken Access Control - OWASP Top 10. https://owasp.org/Top10/A01_2021-Broken_Access_Control/?utm_source=chatgpt.com.

[18] Rapid7. 2024. Metasploit Framework. https://www.metasploit.com.

[19] Abhik Roychoudhury. 2025. Agentic AI for Software: thoughts from Software Engineering community. arXiv:2508.17343 [cs.SE] https://arxiv.org/abs/2508.17343

[20] Xiangmin Shen, Lingzhi Wang, Zhenyuan Li, Yan Chen, Wencheng Zhao, Dawei Sun, Jiashui Wang, and Wei Ruan. 2025. PentestAgent: Incorporating LLM Agents to Automated Penetration Testing. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security, ASIA CCS 2025, Hanoi, Vietnam, August 25-29, 2025*. ACM, 375–391. doi:10.1145/3708821.3733882

[21] Jieke Shi, Zhou Yang, and David Lo. 2025. Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025), 137:1–137:22. doi:10.1145/3708525

[22] Significant Gravitas. 2023. Auto-GPT: An experimental open-source attempt to make GPT-4 fully autonomous. https://github.com/Significant-Gravitas/AutoGPT.

[23] Yaroslav Stefinko, Andrian Piskozub, and Roman Banakh. 2016. Manual and automated penetration testing. Benefits and drawbacks. Modern tendency. In *2016 13th international conference on modern problems of radio engineering, telecommunications and computer science (TCSET)*. IEEE, 488–491.

[24] The Cyphere. 2024. Penetration Testing Statistics, Vulnerabilities and Trends in 2024. https://thecyphere.com/blog/penetration-testing-statistics/.

[25] U.S. Department of the Interior. 2024. Penetration Testing. https://www.doi.gov/ocio/customers/penetration-testing.

[26] Chengran Yang, Zhensu Sun, Hong Jin Kang, Jieke Shi, and David Lo. 2025. Think Like Human Developers: Harnessing Community Knowledge for Structured Code Reasoning. arXiv:2503.14838 [cs.SE] https://arxiv.org/abs/2503.14838

[27] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Julian Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Haoxiang Yang, Aolin Zhang, Rishi Alluri, Nathan Tran, and et al. 2025. Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. https://openreview.net/forum?id=tc90LV0yRL

[28] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. 2025. CVE-Bench: A Benchmark for AI Agents' Ability to Exploit Real-World Web Application Vulnerabilities. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*. OpenReview.net. https://openreview.net/forum?id=3pk0p4NGmQ

[29] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. 2024. Teams of llm agents can exploit zero-day vulnerabilities. *arXiv preprint arXiv:2406.01637* (2024).

[30] Terry Yue Zhuo, Dingmin Wang, Hantian Ding, Varun Kumar, and Zijian Wang. 2025. Cyber-Zero: Training Cybersecurity Agents without Runtime. arXiv:2508.00910 [cs.CR] https://arxiv.org/abs/2508.00910